

ChatGPT integration

Use artificial intelligence to extract information from texts or to generate texts from technical data or other attributes.

Preparation

1. Install Docker (if not installed yet) -> <https://hub.docker.com/editions/community/docker-ce-desktop-mac/>
2. Clone <https://bitbucket.org/blackbitwerbung/pimcore-plugins-data-director> and go to branch "master", refresh it via `git pull`
 - if you do not have git, please download <https://bitbucket.org/blackbitwerbung/pimcore-plugins-data-director/get/master.zip> and unzip it
3. Go to the directory which you cloned / unzipped the files into
4. On command line run `./docker-setup.sh` and wait till this is finished
5. Go to <http://localhost:2000/admin/> -> a Pimcore backend should appear
6. Login with User: admin, Password: admin
7. Right-click on Assets -> Home -> click "Import from Server" and import `/data-director-bundle/examples/`
8. Create object brick **Clothing** based on [its object brick definition](#)
9. Create data object class **Product** and import [its class definition](#)

Topics

1. [Infer technical data from texts](#)
2. [Create texts based on technical data](#)
3. [Modify the prompt](#)

Infer technical data from texts

After creating the target state in your data model, it is often a lot of work to extract this information for your existing data. Artificial intelligence can help to extract information for single fields from textual information.

Especially distributors / resellers often only get product information from their suppliers in an unstructured way.

Use-case: Set up a CSV import and fill object brick fields **material**, **colors** and **gender** based on the product name and description from the CSV file

Input:

- Product name: Wrangler Men's Casual Shirt
- Product description: Long-sleeved cotton shirt with navy and orange stripes

Result:

- Material: cotton
- colors: orange, blue (because the `colors` field does not allow `navy` but it allows `blue`)
- gender: male

This data can now be used for filters or structured exports.

Create texts based on technical data

Especially manufacturers often have the challenge that they know all details of their products exactly but struggle with creating good texts. But also for distributors / resellers it is a big challenge to create good texts because multiple distributors get the same texts from a certain supplier - so how to stand out?

Use-case: Create dataport which uses product name and the technical object brick data to create a product description.

Input:

- Name: Wrangler Men's Casual Shirt
- Material: cotton,
- Colors: Blue, Orange
- Gender: Male

Output:

The Wrangler Men's Casual Shirt is a comfortable and stylish choice for any man. Made from high-quality cotton, it offers breathable comfort that will keep you feeling great all day long. Available in two vibrant colors - blue and orange - this shirt is perfect for casual wear or dressier occasions. Designed with the male physique in mind, it provides a flattering fit that looks great on every body type. Whether you're heading to work or out with friends, the Wrangler Men's Casual Shirt has got you covered!

Modify the prompt

Behind the scenes to configured input data in only one part of the prompt. Some context information like class name, object name etc. will automatically get added. Also limitations (e.g. select options, length limit) get added automatically.

Sometimes it is necessary to modify / extend the prompt.

Use-case: Use youth language to better target your audience.

Input:

- Name: Wrangler Men's Casual Shirt
- Material: cotton,
- Colors: Blue, Orange
- Gender: Male

Use youth language.

Output:

Yo! Check out this sick shirt from Wrangler, it's perfect for dudes who wanna look fly while still keeping it casual. It's made of cotton so you know it'll be comfy af, and comes in two dope colors: blue and orange. Plus, it's designed specifically for guys so you can rock that masculine style with confidence. Cop one today and step up your fashion game!

If you prefer to create the prompt on your own, you can add `<no-prompt-extension>` in the callback function.